

NOTAS A MANO SOBRE
ANÁLISIS DE COMPLEJIDAD
COMPUTACIONAL

Segunda edición

NOTAS A MANO SOBRE ANÁLISIS DE COMPLEJIDAD COMPUTACIONAL

Segunda edición

Autores:

Carlos Eduardo Orozco Garcés

César Jesús Pardo Calvache

Mauro Callejas Cuervo

Capítulo 2

Ejercicios propuestos

Ejercicio 1

Una máquina ejecuta una operación en $T_0 = 0,001$ [s] y consume $S_0 = 8$ [bytes] de memoria por instrucción. Se desea calcular el tiempo total de ejecución t y el consumo total de memoria s para distintas funciones de complejidad, considerando una entrada de tamaño $n = 10^6$ datos. Se estudian tres casos:

1. $T(n) = n, S(n) = n$
2. $T(n) = n \log_2(n), S(n) = \log_2(n)$
3. $T(n) = \sqrt{n}, S(n) = \log_2^2(n)$

Desarrollo:

- **Caso 1:** $T(n) = n, S(n) = n$

En este caso, tanto el número de operaciones como la cantidad de memoria dependen linealmente del tamaño de entrada:

$$t = T_0 \cdot T(n) = 0,001 \cdot 10^6 = 1000 \text{ [s]}$$

$$s = S_0 \cdot S(n) = 8 \cdot 10^6 = 8,0 \cdot 10^6 \text{ [bytes]} = 8,0 \text{ [MB]}$$

- **Caso 2:** $T(n) = n \log_2(n), S(n) = \log_2(n)$

Primero, se aproxima el valor de $\log_2(10^6)$:

$$\log_2(10^6) = \frac{\log_{10}(10^6)}{\log_{10}(2)} = \frac{6}{0,3010} \approx 19,93$$

Luego:

$$t = T_0 \cdot T(n) = 0,001 \cdot 10^6 \cdot 19,93 \approx 19930 \text{ [s]}$$

$$s = S_0 \cdot S(n) = 8 \cdot 19,93 \approx 159,45 \text{ [bytes]}$$

Nota: el uso de memoria en este caso no crece con el tamaño de entrada, sino con su logaritmo.

- **Caso 3:** $T(n) = \sqrt{n}$, $S(n) = \log_2^2(n)$

Primero se calcula la raíz cuadrada de n y el cuadrado del logaritmo base 2:

$$\sqrt{10^6} = 1000, \quad \log_2(10^6) \approx 19,93 \Rightarrow \log_2^2(10^6) \approx 397,2$$

Entonces:

$$t = T_0 \cdot T(n) = 0,001 \cdot 1000 = 1 \text{ [s]}$$

$$s = S_0 \cdot S(n) = 8 \cdot 397,2 \approx 3177,6 \text{ [bytes]} \approx 3,1 \text{ [KB]}$$

Ejercicio 2

Una operación básica toma en promedio $T_0 = 0,002 \text{ [s]}$, y el algoritmo tiene complejidad $T(n) = n^3$. Para una entrada de tamaño $n = 500$, se solicita calcular:

1. el tiempo total de ejecución t ,
2. la cantidad máxima de instrucciones que puede ejecutar la máquina antes de superar un límite de 1 [GB] de memoria, si cada instrucción requiere 16 [bytes].

Desarrollo:

- **Tiempo de ejecución**

La complejidad cúbica para $n = 500$ implica:

$$T(n) = 500^3 = 125 \cdot 10^6 \text{ operaciones}$$

$$t = 0,002 \cdot 125 \cdot 10^6 = 250000 \text{ [s]}$$

Para facilitar su interpretación, se convierte a días:

$$\frac{250000}{86400} \approx 2,89 \text{ días}$$

- **Límite de instrucciones por capacidad de memoria**

Se considera un límite de 1 [GB] = 10^9 [bytes]. Si cada instrucción requiere 16 [bytes]:

$$\text{Máximo de instrucciones} = \frac{10^9}{16} = 62,5 \cdot 10^6$$

Comentario: El número total de instrucciones que se pretende ejecutar es $125 \cdot 10^6$, lo cual supera en el doble la capacidad de la memoria disponible.

Ejercicio 3

Un algoritmo tiene complejidad temporal $T(n) = 2^n$ y consumo de memoria $S(n) = n!$. Se solicita calcular el tiempo de ejecución y consumo de memoria para $n \in \{1, 10, 1000, 10000\}$, con $T_0 = 0,0005$ [s] y $S_0 = 4$ [bytes].

Desarrollo:

- Para $n = 1$:

$$\begin{aligned} T(n) &= 2^1 = 2 \\ t &= 0,0005 \cdot 2 = 0,001 \text{ [s]} \\ S(n) &= 1! = 1 \\ s &= 4 \cdot 1 = 4 \text{ [bytes]} \end{aligned}$$

- Para $n = 10$:

$$\begin{aligned} T(n) &= 2^{10} = 1024 \\ t &= 0,0005 \cdot 1024 = 0,512 \text{ [s]} \\ S(n) &= 10! = 3628800 \\ s &= 4 \cdot 3628800 = 14515200 \text{ [bytes]} = 13,84 \text{ [MB]} \end{aligned}$$

- Para $n = 1000$ y $n = 10000$:

$$\begin{aligned} T(1000) &= 2^{1000} \gg 10^{300}, \quad S(1000) = 1000! \\ T(10000) &= 2^{10000} \gg 10^{3000}, \quad S(10000) = 10000! \end{aligned}$$

Comentario: Los valores para $n = 1000$ y superiores son tan grandes que ningún sistema actual puede ejecutar o almacenar semejante cantidad de operaciones o datos.

Ejercicio 4

Un sistema debe procesar $n = 10^6$ datos en menos de 10 [s], y cada instrucción toma $T_0 = 0,001$ [s]. Determinar qué funciones de complejidad temporal permiten cumplir esta restricción.

Desarrollo:

La cantidad máxima de operaciones permitidas es:

$$T(n) \leq \frac{10}{0,001} = 10^4$$

Evaluamos diferentes funciones en $n = 10^6$:

- $T(n) = \log_2(n) \Rightarrow \log_2(10^6) \approx 19,93$

- $T(n) = \log_2^2(n) \approx 397,2$
- $T(n) = \sqrt{n} = 1000$
- $T(n) = n = 10^6 \gg 10^4$

Comentario: Solo funciones sublineales como $\sqrt{n}$ o $\log^2(n)$ permiten cumplir el límite de 10 segundos.

Ejercicio 5

Se comparan tres algoritmos:

- $A_1(n) : T(n) = n \log_2(n)$
- $A_2(n) : T(n) = n^2$
- $A_3(n) : T(n) = 10n^2$

Con $T_0 = 0,002$ [s], se solicita:

1. Calcular el tiempo de ejecución para $n = 10^5$
2. Comparar los tres algoritmos para $n = 5000$

Desarrollo:

- Para $n = 10^5$:

$$\log_2(10^5) \approx \frac{5}{0,3010} \approx 16,61$$

$$T_1 = 10^5 \cdot 16,61 = 1,661 \cdot 10^6$$

$$t_1 = 0,002 \cdot 1,661 \cdot 10^6 = 3322 \text{ [s]}$$

$$T_2 = (10^5)^2 = 10^{10}$$

$$t_2 = 0,002 \cdot 10^{10} = 2 \cdot 10^7 \text{ [s]} \approx 231 \text{ [días]}$$

$$T_3 = 10 \cdot 10^{10} = 10^{11}$$

$$t_3 = 0,002 \cdot 10^{11} = 2 \cdot 10^8 \text{ [s]} \approx 2315 \text{ [días]}$$

- Para $n = 5000$:

$$\log_2(5000) \approx 12,29$$

$$T_1 = 5000 \cdot 12,29 = 61450$$

$$t_1 = 0,002 \cdot 61450 = 122,9 \text{ [s]}$$

$$T_2 = 5000^2 = 25 \cdot 10^6$$

$$t_2 = 0,002 \cdot 25 \cdot 10^6 = 50000 \text{ [s]}$$

$$T_3 = 10 \cdot 25 \cdot 10^6 = 250 \cdot 10^6$$

$$t_3 = 0,002 \cdot 250 \cdot 10^6 = 500000 \text{ [s]}$$

Comentario: La diferencia en rendimiento es notable incluso para valores pequeños de n . A_1 ofrece tiempos razonables, mientras que A_2 y A_3 son ineficaces.

Ejercicio 6

Se desea procesar $n = 10^7$ datos en menos de 5 [s], y la máquina ejecuta en promedio una instrucción en $T_0 = 10^{-7}$ [s]. Determinar qué funciones de complejidad temporal permiten cumplir esta restricción.

Desarrollo:

$$T(n) \leq \frac{5}{10^{-7}} = 5 \cdot 10^7$$

$$\blacksquare T(n) = n \Rightarrow T(10^7) = 10^7$$

$$t = 10^{-7} \cdot 10^7 = 1 \text{ [s]}$$

$$\blacksquare T(n) = n \log_2(n) \Rightarrow \log_2(10^7) \approx 23,25$$

$$T(n) \approx 10^7 \cdot 23,25 = 2,325 \cdot 10^8$$

$$t = 10^{-7} \cdot 2,325 \cdot 10^8 = 23,25 \text{ [s]}$$

Comentario: Solo funciones como $T(n) = n$ o menores permiten cumplir el tiempo límite.

Ejercicio 7

Por restricciones físicas, una máquina solo cuenta con 128 [MB] de memoria disponible. Se desea determinar qué complejidad espacial permitiría procesar $n = 10^5$ datos sin exceder ese límite.

Desarrollo:

Convertimos la memoria total disponible a bytes:

$$128 \cdot 2^{20} = 134,217,728 \text{ [bytes]}$$

Calculamos cuántos bytes están disponibles por cada dato:

$$\frac{134,217,728}{10^5} = 1344 \text{ [bytes por dato]}$$

Esto implica que cualquier función de complejidad espacial $S(n)$ debe cumplir:

$$S(n) \cdot c \leq 134,217,728$$

- $S(n) = n$:

$$s = c \cdot n = c \cdot 10^5 \Rightarrow c \leq \frac{134,217,728}{10^5} = 1344 \text{ [bytes]}$$

- $S(n) = n \log_2(n)$:

$$\log_2(10^5) \approx 16,61$$

$$S(n) \approx 10^5 \cdot 16,61 = 1,661,000$$

$$c \leq \frac{134,217,728}{1,661,000} \approx 80,8 \text{ [bytes]}$$

- $S(n) = n^2$:

$$S(n) = 10^{10}, \quad s = c \cdot 10^{10} \Rightarrow c \leq \frac{134,217,728}{10^{10}} = 0,013 \text{ [bytes]}$$

Comentario: Funciones como $S(n) = n$ y $S(n) = n \log(n)$ son aceptables si las estructuras de datos utilizadas no exceden 1344 o 81 bytes respectivamente. Funciones cuadráticas o mayores no son admisibles bajo esta restricción de memoria.

Ejercicio 8

Estimar el tiempo que tomaría procesar $n = 10^6$ datos con un algoritmo cuya complejidad es $T(n) = n!$, y cada instrucción toma $T_0 = 10^{-5}$ [s].

Desarrollo:

$$T(10^6) = 10^6! \gg 10^{5,5 \cdot 10^6}$$

$$t = 10^{-5} \cdot 10^6! \Rightarrow \text{inviable}$$

Comentario: El tiempo de ejecución supera cualquier escala temporal realista. Este resultado solo muestra nuevamente que los algoritmos con complejidad factorial son imprácticos en la mayoría de los casos.

Ejercicio 9

Un algoritmo con complejidad espacial $S(n) = n \log_2(n)$ utiliza 16 bytes por unidad de almacenamiento. Determinar cuánto espacio de memoria consume al recibir una entrada de tamaño $n = 10000$, expresando el resultado en kilobytes (KB) y kibibytes (KiB).

Desarrollo:

$$\log_2(10000) \approx \frac{\log_{10}(10000)}{\log_{10}(2)} = \frac{4}{0,3010} \approx 13,29$$

$$S(10000) = 10000 \cdot 13,29 = 132900$$

$$s = 132900 \cdot 16 = 2126400 \text{ [bytes]}$$

$$\text{KB} = \frac{2126400}{1000} = 2126,4 \text{ [KB]}$$

$$\text{KiB} = \frac{2126400}{1024} \approx 2076,56 \text{ [KiB]}$$

Comentario: La diferencia entre kilobytes y kibibytes proviene del sistema de medida:

- El kilobyte (KB) sigue el sistema decimal: $1 \text{ [KB]} = 1000 \text{ [bytes]}$.
- El kibibyte (KiB) sigue el sistema binario: $1 \text{ [KiB]} = 1024 \text{ [bytes]}$.

Aunque la diferencia es pequeña para valores bajos, puede volverse significativa en sistemas con restricciones de almacenamiento.

Ejercicio 10

Calcular cuánto tiempo tomaría procesar $n = 20$ datos con un algoritmo de complejidad $T(n) = 2^n$, si cada instrucción toma 1 [s].

Desarrollo:

$$T(20) = 2^{20} = 1,048,576$$

$$t = 1 \cdot 1,048,576 = 1,048,576 \text{ [s]} = \frac{1,048,576}{86400} \approx 12,13 \text{ [días]}$$

Ejercicio 11

Un algoritmo con complejidad temporal $T(n) = n^2$ tarda 0,01 [s] en procesar $n = 1000$ datos. Estimar el tiempo necesario para procesar $n = 100000$ datos, asumiendo que el comportamiento se mantiene estable.

Desarrollo:

Calculamos el número de operaciones para $n = 1000$:

$$T(1000) = 1000^2 = 10^6$$

Dado que eso toma 0,01 [s], el tiempo por operación es:

$$T_0 = \frac{0,01}{10^6} = 10^{-8} \text{ [s]}$$

Ahora calculamos las operaciones para $n = 100000$:

$$T(100000) = (10^5)^2 = 10^{10}$$

$$t = T_0 \cdot T(n) = 10^{-8} \cdot 10^{10} = 100 \text{ [s]}$$

Ejercicio 12

Se desea implementar un algoritmo que procese $n = 10000$ datos en menos de 1 [s]. La máquina ejecuta 10^7 instrucciones por segundo. Identificar qué órdenes de complejidad temporal permiten cumplir esta restricción.

Desarrollo:

Número máximo de operaciones permitidas:

$$T(n) \leq 10^7$$

Evaluamos distintas funciones:

- $T(n) = n \Rightarrow 10000 \leq 10^7$: válida
- $T(n) = n \log_2(n)$

$$\log_2(10000) \approx 13,29$$

$$T(n) = 10000 \cdot 13,29 = 132900$$

Como $132900 < 10^7$: también válida

- $T(n) = n^2 = 10^8 > 10^7$: no válida

Ejercicio 13

Un sistema de distribución utiliza un algoritmo con complejidad espacial $S(n) = 2^n$. Puede procesar $n = 20$ datos. Determinar cuánta memoria se requiere para procesar $n = 25$, expresando el resultado en una unidad adecuada.

Desarrollo:

$$S(25) = 2^{25} = 33554432 \text{ [unidades]}$$

Si cada unidad ocupa 1 [byte]:

$$s = 33554432 \text{ [bytes]}$$

Convertimos a MiB:

$$\text{MiB} = \frac{33554432}{2^{20}} = \frac{33554432}{1048576} = 32 \text{ [MiB]}$$

Ejercicio 14

Un sistema embebido es capaz de realizar 10^6 operaciones por segundo y dispone de 1 [MB] de memoria disponible. Se solicita determinar qué órdenes de complejidad temporal y espacial permitirían procesar $n = 5000$ datos de forma eficiente.

Desarrollo:

Análisis temporal:

El número máximo de operaciones permitidas por segundo es:

$$T(n) \leq 10^6$$

Evaluamos distintas funciones para $n = 5000$:

- $T(n) = n = 5000 \Rightarrow t = \frac{5000}{10^6} = 0,005$ [s]
- $T(n) = n \log_2(n)$

$$\log_2(5000) \approx 12,29$$

$$T(n) = 5000 \cdot 12,29 = 61450$$

$$t = \frac{61450}{10^6} \approx 0,061$$
 [s]

- $T(n) = n^2 = 25 \cdot 10^6 \Rightarrow t = \frac{25 \cdot 10^6}{10^6} = 25$ [s]

Resultado temporal: válidas $T(n) = n$ y $T(n) = n \log n$; la función cuadrática excede el umbral práctico de ejecución inmediata.

Análisis espacial:

Memoria disponible:

$$1 \text{ [MB]} = 2^{20} = 1048576 \text{ [bytes]}$$

Memoria disponible por dato:

$$\frac{1048576}{5000} \approx 209,71 \text{ [bytes]}$$

Análisis por función:

- $S(n) = n \Rightarrow s = c \cdot n = 5000c$

$$c \leq \frac{1048576}{5000} = 209,71 \text{ [bytes]}$$

- $S(n) = n \log_2(n) = 61450$

$$s = c \cdot 61450 \Rightarrow c \leq \frac{1048576}{61450} \approx 17,06 \text{ [bytes]}$$

$$\blacksquare S(n) = n^2 = 25 \cdot 10^6$$

$$s = c \cdot 25 \cdot 10^6 \Rightarrow c \leq \frac{1048576}{25 \cdot 10^6} \approx 0,0419 \text{ [bytes]}$$

Comentario: Bajo estas condiciones, las funciones espaciales aceptables son las de tipo lineal o logarítmico-lineal, siempre que cada unidad no supere los 210 o 17 bytes, respectivamente.

Ejercicio 15

Un algoritmo con complejidad temporal $T(n) = n \log_2(n)$ tarda 2 [s] en procesar 1000 datos. Estimar si puede procesar 10000 datos en menos de 10 [s].

Desarrollo:

$$T(1000) = 1000 \cdot \log_2(1000) \approx 1000 \cdot 9,97 = 9970$$

$$T_0 = \frac{2}{9970} \approx 2,006 \cdot 10^{-4} \text{ [s]}$$

$$T(10000) = 10000 \cdot \log_2(10000) \approx 10000 \cdot 13,29 = 132900$$

$$t = T_0 \cdot T(10000) = 2,006 \cdot 10^{-4} \cdot 132900 \approx 26,66 \text{ [s]}$$

Comentario: El algoritmo no cumple la restricción. Aumentar el tamaño de entrada diez veces casi triplica el tiempo debido al factor logarítmico.